

CM 6 : Arbres

Info1.Algo1

2025-2026 Printemps

1 Introduction

- Notion d'arbre
- Motivations

2 Arbres binaires

- Principe général
- Description du type
- Implémentation
- Écriture de fonctions

3 Parcours

- Parcours en profondeur
- Parcours en largeur

Définition informelle

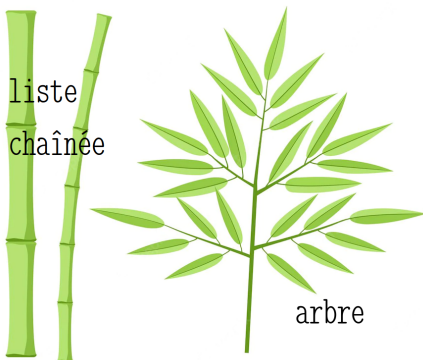
Un **arbre** est une **structure de données récursive**. Chaque élément de cette structure est appelé **nœud**. Chaque nœud est constitué :

- d'une **valeur** associée au nœud,
- d'un nombre quelconque de références vers **un ou plusieurs descendants** ou sous-arbres.

Le premier nœud (qui n'est le descendant d'aucun autre nœud) est appelé **racine** de l'arbre.

Notion d'arbre

La notion d'**arbre** généralise la notion de **liste chaînée** vue précédemment :



Haiku :

*Parmi les arbres
Un bambou
Liste chaînée*

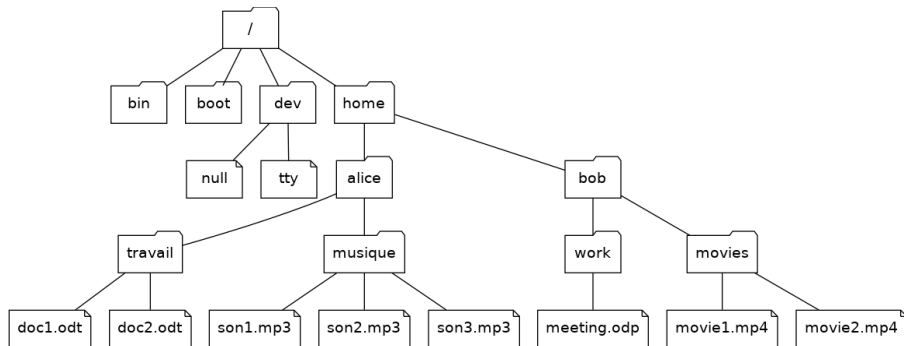
Là où chaque **cellule d'une liste chaînée** ne peut avoir **qu'une seule queue**, chaque **noeud d'un arbre** peut avoir **un ou plusieurs descendants**.

Motivations

Motivation 1 : dans un système d'exploitation

Le **système de fichiers** ainsi que l'**organisation des processus** en cours d'exécution sont tous deux représentés sous forme d'**arbre**.

Exemple 1.1 : système de fichiers



Exemple 1.2 : organisation des processus

Sur un système Linux, les processus en cours d'exécution sont accessibles via la commande `ps`.

```
alexander@alexander-vu:~$ pstree -np
systemd(1)─systemd-journal(206)
           └─systemd-udevd(226)
           └─systemd-timesyn(373)─{sd-resolve}(391)
           └─cupsd(663)─dbus(739)
                     └─dbus(740)
           └─acpid(679)
           └─anacron(682)
           └─rsyslogd(688)─{in:imuxsock}(726)
                        └─{in:imklog}(727)
                        └─{rs:main Q:Reg}(728)
           └─accounts-daemon(694)─{gmain}(708)
                                └─{gdbus}(741)
           └─snapd(696)─{snapd}(722)
                     └─{snapd}(725)
                     └─{snapd}(730)
                     └─{snapd}(734)
```

Motivations

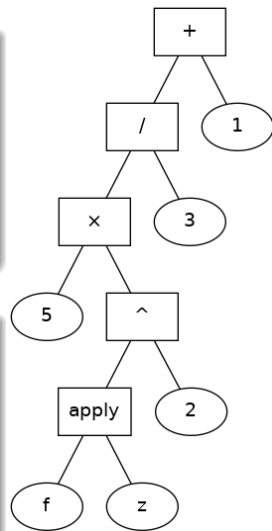
Motivation 2 : arbres syntaxiques

Lors de l'analyse syntaxique d'un langage (formules algébriques ou logiques, langage de programmation,...), l'analyseur produit un **arbre syntaxique** qui représente la structure syntaxique de ce qui a été lu.

Exemple 2.1

L'arbre syntaxique ci-contre représente la formule mathématique :

$$\frac{5 f(z)^2}{3} + 1$$

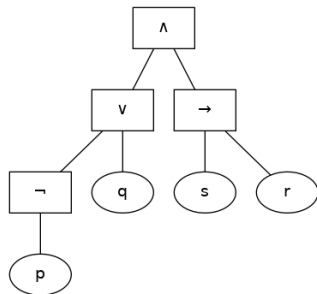


Motivations

Exemple 2.2

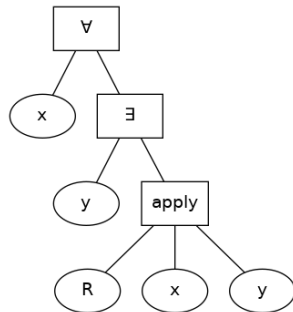
Arbre syntaxique de la formule de la **logique des propositions** :

$$(\neg p \vee q) \wedge (s \rightarrow r)$$



Arbre syntaxique de la formule de la **logique des prédicats** :

$$\forall x, \exists y, R(x, y)$$



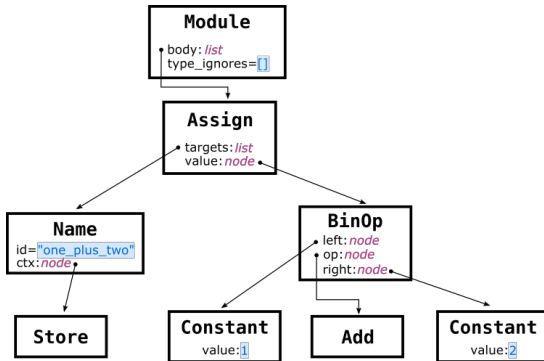
(Cf. UE Info1.DS1 - Structures discrètes 1)

Motivations

Exemple 2.3

L'arbre syntaxique ci-dessous représente le code suivant :

```
1 one_plus_two = 1+2
```



Motivations

Exemple 2.3 (suite)

L'arbre syntaxique peut être obtenu avec le module ast :

```
1 import ast
2 print(ast.dump(ast.parse("one_plus_two = 1+2"),indent=4))
```

```
Module(
  body=[
    Assign(
      targets=[
        Name(id='one_plus_two', ctx=Store())],
      value=BinOp(
        left=Constant(value=1),
        op=Add(),
        right=Constant(value=2))),
    type_ignores=[])
```

Motivation 3 : en algorithmique

La structure de données arbre est aussi un **puissant outil** pour la résolution de certains problèmes (*non nécessairement exprimés sous forme d'arbre*).

Problème posé	Arbre utilisé
Tri d'un tableau	Tas (<i>en anglais : heap</i>)
Représentation d'un tableau associatif	Arbre binaire de recherche Treap, AVL, Arbre bicolore B-arbre, Arbre splay...

1 Introduction

- Notion d'arbre
- Motivations

2 Arbres binaires

- Principe général
- Description du type
- Implémentation
- Écriture de fonctions

3 Parcours

- Parcours en profondeur
- Parcours en largeur

Définition

Un **arbre binaire** est :

- soit un **arbre vide**,
- soit un **noeud** ayant **exactement 2 descendants** (**gauche** et **droite**) qui sont eux-mêmes des **arbres binaires**.

L'un ou l'autre des descendants peut être **vide**, ce qui permet de représenter :

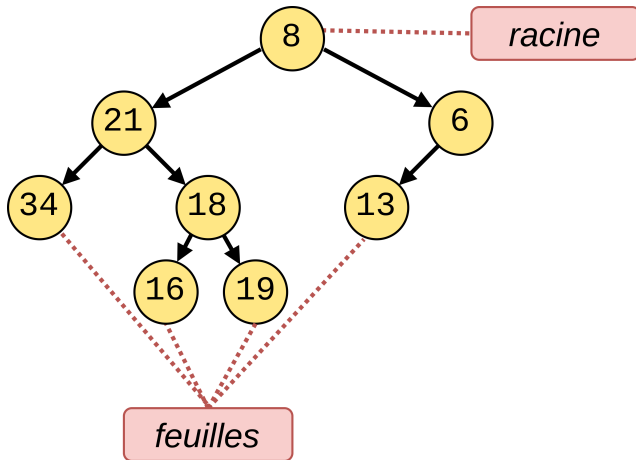
- Les **nœuds** ayant **un seul descendant non vide**.
- Les **feuilles** (ayant deux descendants vides).

Chaque noeud est **porteur d'une information** (un entier, une chaîne, ...).

L'unique noeud qui n'a pas de parent est la **racine**.

Principe général

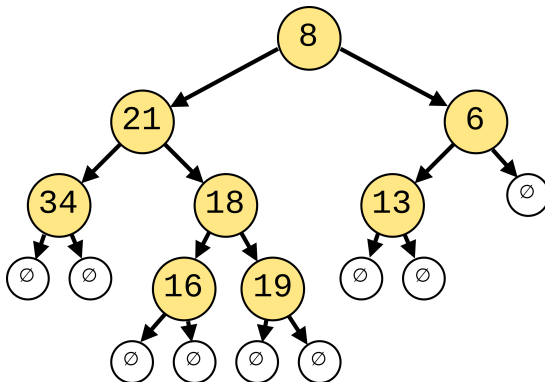
Exemple



Principe général

Exemple (suite)

Avec tous les noeuds vides :



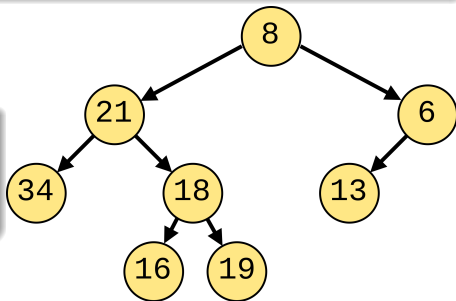
Principe général

Caractéristiques

- On appelle **taille** d'un arbre le nombre de nœuds de cet arbre.
- On appelle **hauteur** d'un arbre le nombre de nœuds du plus long chemin entre la racine et une feuille quelconque de l'arbre.
La hauteur d'un **arbre vide** est 0.

Exemple (suite)

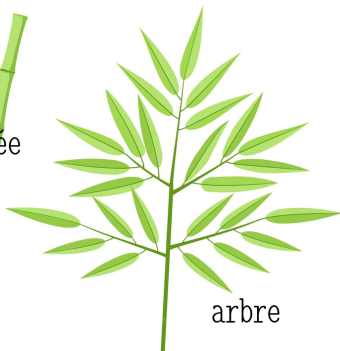
- **taille** : 8
- **hauteur** : 4



Principe général

Principe général

- Toutes les valeurs associées aux noeuds sont de **même type**.
- C'est une structure **réursive** : on distingue l'élément **racine** et le reste (son **sous-arbre gauche** et son **sous-arbre droit**).



On applique une démarche semblable à celle adoptée pour les listes chaînées.

Opérations autorisées

- Opérations de construction
 - Création d'un arbre vide.
 - Création d'un arbre à partir de la valeur de sa racine et de ses deux sous-arbres (gauche et droit).
- Opérations d'accès
 - Accès à la valeur associée à la racine.
 - Accès aux sous-arbres gauche et droit.
 - Test d'arbre vide.

Spécification des fonctions d'interface

Les opérations autorisées seront accessibles via les seules fonctions suivantes :

- Opérations de création
 - `creer_arbre_vide()` : retourne un arbre vide.
 - `creer_arbre(r,g,d)` : retourne l'arbre constitué de la racine `r` et des sous-arbres gauche `g` et droit `d`.
- Opérations d'accès
 - `racine(arbre)` : retourne la valeur racine de l'arbre passé en paramètre.
 - `gauche(arbre)` et `droite(arbre)` : retournent respectivement les sous-arbres gauche et droit de l'arbre passé en paramètre.
 - `est_vide(arbre)` : retourne `True` si l'arbre passé en paramètre est vide, `False` sinon.

Principe

Représentation à l'aide du type `tuple` :

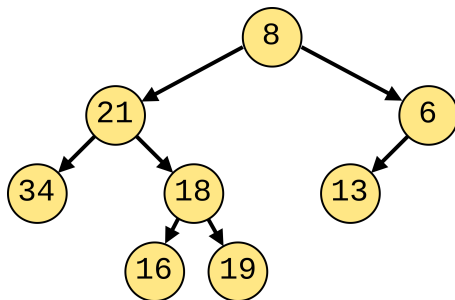
- un arbre vide est représenté par la valeur `None`
- un arbre non vide par un tuple de trois éléments :
 - le premier est la valeur associée à la **racine** (`arbre[0]`)
 - le deuxième est le **sous-arbre gauche** (`arbre[1]`)
 - le troisième est le **sous-arbre droit** (`arbre[2]`)

Remarques :

- On ne peut pas modifier un tuple $\Rightarrow$ sécurisation du type.
- On pourra changer de convention plus tard (exemple : représenter l'arbre vide par un tuple vide).
Cela **ne doit en aucun cas** changer quoi que ce soit aux autres fonctions qui utiliseront ces fonctions d'interface.

Implémentation

Exemple (suite)

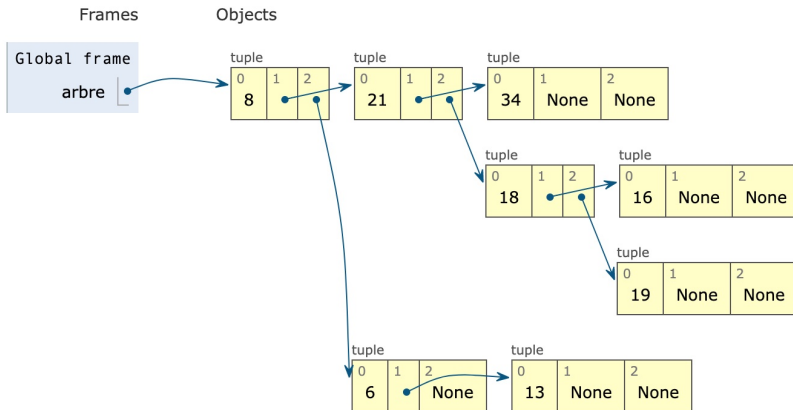


Représentation de l'arbre donné en exemple :

```
arbre = (8, (21, (34, None, None), (18, (16, None, None), (19, None, None))), (6, (13, None, None), None))
```

Implémentation

Exemple (suite)



Visualiser avec PythonTutor

Implémentation

Détails de l'implémentation

L'implémentation particulière choisie ici est la suivante :

- Un arbre vide est représentée par `None`
- Un arbre non vide est un tuple `(r,g,d)`

```
def creer_arbre_vide():  
    return None  
  
def creer_arbre(r,g,d):  
    return r,g,d  
  
def racine(arbre):  
    return arbre[0]  
  
def gauche(arbre):  
    return arbre[1]  
  
def droite(arbre):  
    return arbre[2]  
  
def est_vide(arbre):  
    return arbre==None
```

Autres implémentations possibles

- pas de module standard en Python (mais on trouve divers modules à installer comme `tree`)
- tuple ou objet en Python, objet en JAVA, ...
- struct plus pointeurs en C
- tableaux dynamiques

Écriture de fonctions

La récursivité est particulièrement adaptée pour traiter des structures de données récursives comme les arbres

Méthodologie

- **Cas d'arrêt :**
 - En général, arbre vide.
 - Éventuellement feuille : ses deux sous-arbres sont vides.
- **Cas récursif :**
 - En général, appeler la fonction **récursivement sur les deux sous-arbres**.
 - Éventuellement sur **un seul sous-arbre** : problème dissymétrique ou élément déjà trouvé sur le premier sous-arbre exploré, Arbres Binaires de Recherche (ABR)...
 - **On se convainc que la fonction termine** : on a diminué la taille de l'arbre en passant aux sous-arbres.

Écriture de fonctions

Exemple 1 (retournant un entier)

Écrire la fonction récursive `calculer_taille` qui accepte en paramètre un arbre et renvoie sa **taille**.

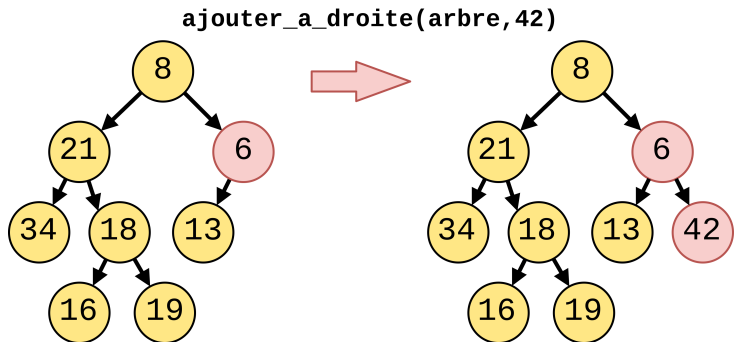
Écrire la fonction récursive `calculer_hauteur` qui accepte en paramètre un arbre et renvoie sa **hauteur**.

Exemple 2 (retournant un arbre)

Écrire la fonction récursive `ajouter_a_droite` qui accepte en paramètres un arbre ainsi qu'un entier `valeur` et retourne un nouvel arbre résultat de l'ajout d'une feuille à l'arbre donné en paramètre. Cette feuille sera ajoutée le plus à droite de l'arbre, et aura pour valeur l'entier passé en paramètre.

Écriture de fonctions

Dans l'arbre traité en exemple, le noeud le plus à droite a pour valeur 6, et on lui ajoute une feuille à sa droite :



1 Introduction

- Notion d'arbre
- Motivations

2 Arbres binaires

- Principe général
- Description du type
- Implémentation
- Écriture de fonctions

3 Parcours

- Parcours en profondeur
- Parcours en largeur

2 types de parcours

Lors d'un traitement, on doit déterminer l'**ordre** selon lequel les différents nœuds sont examinés, ce qui conditionne l'ordre de traitement des informations.

Deux politiques de parcours sont possibles :

- Parcours en **profondeur** (avec différentes variantes).
- Parcours en **largeur**.

Parcours en profondeur

Parcours en profondeur

Lors d'un **parcours en profondeur préfixe** d'un arbre :

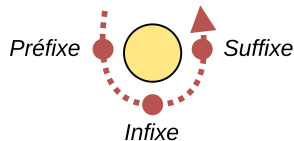
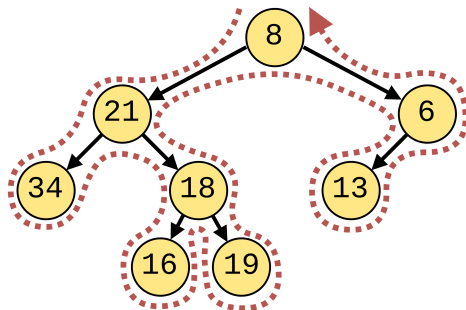
- On traite d'abord la **racine** de l'arbre.
- On effectue un parcours en profondeur préfixe du sous-arbre **gauche**, puis **droit**.

De même on peut définir les **parcours en profondeur infixes et suffixes**.

type de parcours en profondeur	ordre de traitement
préfixe	racine gauche droite
infixe	gauche racine droite
suffixe	gauche droite racine

Parcours en profondeur

Exemple (suite)



type de parcours en profondeur	ordre de traitement
préfixe	8 21 34 18 16 19 6 13
infixe	34 21 16 18 19 8 13 6
suffixe	34 16 19 18 21 13 6 8

Exemple

Écrire les fonctions récursives `parcours_prefixe`, `parcours_infixe` et `parcours_suffixe` qui acceptent en paramètre un arbre et **affichent** les valeurs des noeuds de cet arbre selon un parcours en profondeur préfixe, infixe et suffixe.

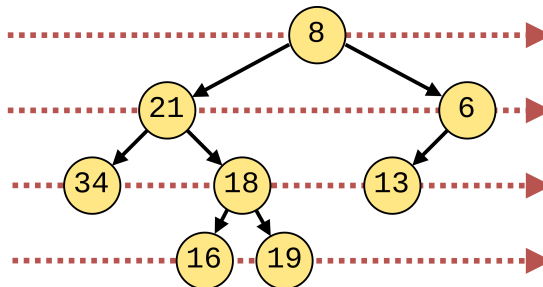
Parcours en largeur

Lors d'un **parcours en largeur** d'un arbre, on traverse l'arbre **niveau par niveau**, en partant du niveau 0 jusqu'au niveau le plus profond.

Pour chaque niveau, le parcours des nœuds s'effectue de la gauche vers la droite.

Parcours en largeur

Exemple (suite)



Ordre de traitement pour le parcours en largeur :

8 21 6 34 18 13 16 19

Algorithme du parcours en largeur : principes généraux

On souhaite écrire la fonction `parcours_largeur` qui accepte en paramètre un arbre et affiche la valeur des noeuds de cet arbre selon un parcours en largeur.

- Contrairement à ce que l'on pourrait croire, cette fonction n'est **pas récursive**.
- L'écriture d'une fonction de parcours en largeur nécessite une structure de données **file** qui sera vue ultérieurement.
- Pour l'instant nous utilisons une **liste native Python** (de type `list`) **contenant des arbres**.
 - Les arbres sont **ajoutés en fin** : méthode `append(...)`
 - Les arbres à traiter sont **retirés en début** : méthode `.pop(0)`

Algorithme du parcours en largeur : détails

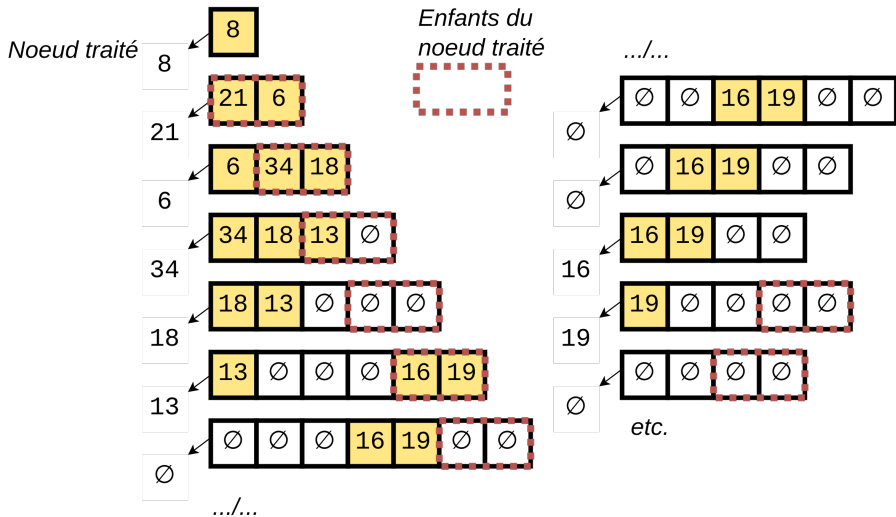
La liste native `arbres_a_traiter` est initialisée avec l'arbre passé en paramètre. On va y placer tous les sous-arbres dont on souhaite temporiser le traitement.

L'algorithme est basé sur une boucle `while` qui s'arrête lorsque la liste `arbres_a_traiter` est vide.

À chaque itération de cette boucle :

- On retire l'arbre courant à traiter.
- Si cet arbre est non vide :
 - On affiche la valeur de la racine.
 - On ajoute ses deux sous-arbres dans la liste `arbres_a_traiter`.

Parcours en largeur

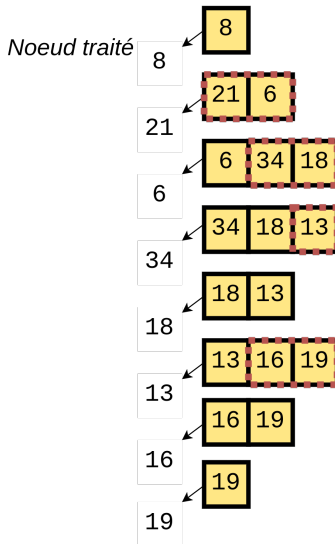


Parcours en largeur

```
1 def parcours_largeur(arbre):  
2     arbres_a_traiter = [arbre]  
3     while len(arbres_a_traiter)>0:  
4         arbre_courant = arbres_a_traiter.pop(0)  
5         if not est_vide(arbre_courant):  
6             print(racine(arbre_courant),end=' ')  
7             arbres_a_traiter.append(gauche(arbre_courant))  
8             arbres_a_traiter.append(droite(arbre_courant))
```

Parcours en largeur

Remarque : on peut optimiser cet algorithme en ne rajoutant les noeuds en fin de liste que lorsque ceux-ci sont non vides.



Enfants du
noeud traité



Parcours en largeur

Dans cette version on ne place jamais un arbre vide dans la liste `arbres_a_traiter` :

```
1 def parcours_largeur(arbre):
2     if est_vide(arbre):
3         return
4     arbres_a_traiter = [arbre]
5     while len(arbres_a_traiter)>0:
6         arbre_courant = arbres_a_traiter.pop(0)
7         print(racine(arbre_courant),end=' ')
8         if not est_vide(gauche(arbre_courant)):
9             arbres_a_traiter.append(gauche(arbre_courant))
10        if not est_vide(droite(arbre_courant)):
11            arbres_a_traiter.append(droite(arbre_courant))
```

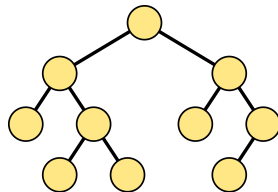
Parcours en largeur

Applications du parcours en largeur

Les **arbres** forment un type particulier de **graphes**.

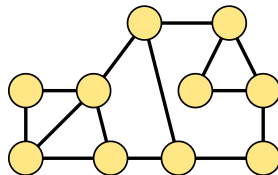
L'algorithme de **parcours en largeur** (*Breadth-First Search* ou *BFS*) se généralise aux **graphes** et a dans ce cadre de nombreuses applications.

Il garantit de trouver le **chemin le plus court** (en nombre d'arêtes) entre un noeud source et un autre noeud.



ARBRE

GRAPHE

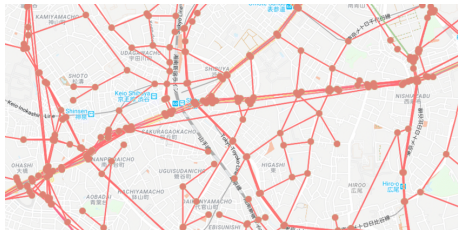


Parcours en largeur

Applications du parcours en largeur

Recherche de distance minimale

Guidage routier :



Déplacements dans un jeu vidéo :



Parcours en largeur

Applications du parcours en largeur

- Parcours de réseaux : informatiques, sociaux...
- Exploration de sites web : Wiki Game,
<https://www.sixdegreesofwikipedia.com/>

